

# Data Structures in Java

Java-Oriented Study Notes • Concepts + Operations + Code + Complexity

**Goal:** Learn the important data structures and implement their basic operations using Java.

| Unit | Topics                            |
|------|-----------------------------------|
| 1    | Basics, ADT, Complexity, Arrays   |
| 2    | Linked Lists                      |
| 3    | Stacks and Queues                 |
| 4    | Trees and Binary Search Trees     |
| 5    | Heaps and Priority Queue          |
| 6    | Hashing / HashMap / HashSet       |
| 7    | Graphs and Traversals             |
| 8    | Searching and Sorting             |
| 9    | Java Collections Framework        |
| 10   | Quick Revision + Complexity Table |

# 1. Data Structures Fundamentals

## What is a Data Structure?

A data structure is a way of organizing and storing data so that it can be accessed and modified efficiently. Examples include arrays, linked lists, stacks, queues, trees, graphs, and hash tables.

## Abstract Data Type (ADT)

An ADT describes what operations are supported without specifying how they are implemented. For example, a Stack ADT supports push, pop, peek, and isEmpty. It can be implemented using an array or linked list.

## Time Complexity

- **O(1)** – constant time; operation does not depend on input size.
- **O(log n)** – logarithmic; common in binary search and balanced tree operations.
- **O(n)** – linear; one pass through n elements.
- **O(n log n)** – common in efficient comparison sorting such as merge sort.
- **O(n<sup>2</sup>)** – quadratic; common in simple nested-loop sorting such as bubble sort.

## Arrays in Java

Java arrays have fixed size and store elements of the same type.

```
int[] marks = {85, 90, 78, 92};

System.out.println(marks[0]);

for (int i = 0; i < marks.length; i++) {
    System.out.println(marks[i]);
}
```

- Access by index: O(1).
- Search in an unsorted array: O(n).
- Insertion/deletion in the middle: O(n) because elements may need to shift.
- Useful when size is known and fast indexed access is required.

## 2. Linked Lists

A linked list consists of nodes. Each node stores data and a reference to another node. Unlike an array, linked-list elements are not required to be stored in contiguous memory.

### Singly Linked List

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

class SinglyLinkedList {
    Node head;

    void insertAtBeginning(int data) {
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
    }

    void display() {
        Node temp = head;
        while (temp != null) {
            System.out.print(temp.data + " -> ");
            temp = temp.next;
        }
        System.out.println("null");
    }
}
```

### Types

- **Singly Linked List:** each node points to the next node.
- **Doubly Linked List:** each node has next and previous references.
- **Circular Linked List:** the last node points back to the first node.

### Advantages and Limitations

- Dynamic size.
- Insertion/deletion at a known node or beginning can be efficient.
- Random access is not efficient; reaching an element generally requires traversal.
- Extra memory is needed for references.

## 3. Stack and Queue

### Stack – LIFO

Stack follows **Last In, First Out (LIFO)**. The most recently inserted element is removed first.

```
Stack<Integer> stack = new Stack<>();

stack.push(10);
stack.push(20);
stack.push(30);

System.out.println(stack.peek()); // 30
System.out.println(stack.pop());  // 30
```

- push(x): insert an element.

- `pop()`: remove the top element.
- `peek()`: view the top element.
- `isEmpty()`: check whether the stack is empty.
- Applications: function calls, undo operations, expression processing, DFS.

## Queue – FIFO

Queue follows **First In, First Out (FIFO)**. The first inserted element is removed first.

```
Queue<Integer> q = new LinkedList<>();
```

```
q.offer(10);
q.offer(20);
q.offer(30);
```

```
System.out.println(q.peek()); // 10
System.out.println(q.poll()); // 10
```

- `offer(x)`: insert at the rear.
- `poll()`: remove from the front.
- `peek()`: view the front.
- Applications: scheduling, buffering, BFS, printer/task queues.

## 4. Trees and Binary Search Trees

### Tree Terminology

- **Root:** top node.
- **Parent / Child:** direct relationship between nodes.
- **Leaf:** node with no children.
- **Height:** length of the longest path from a node to a leaf, depending on convention.
- **Subtree:** a node together with its descendants.

### Binary Tree

A binary tree is a tree in which each node has at most two children: left and right.

### Binary Search Tree (BST)

For a standard BST, values smaller than a node are stored in its left subtree and larger values in its right subtree. The exact handling of duplicates depends on the implementation.

```
class TreeNode {
    int data;
    TreeNode left, right;

    TreeNode(int data) {
        this.data = data;
    }
}

TreeNode insert(TreeNode root, int value) {
    if (root == null)
        return new TreeNode(value);

    if (value < root.data)
        root.left = insert(root.left, value);
    else if (value > root.data)
        root.right = insert(root.right, value);

    return root;
}

void inorder(TreeNode root) {
    if (root != null) {
        inorder(root.left);
        System.out.print(root.data + " ");
        inorder(root.right);
    }
}
```

**Important:** Inorder traversal of a BST produces the stored keys in sorted order when the BST follows the usual ordering rule.

## 5. Heaps and Priority Queue

A heap is a complete binary tree commonly used for priority-based processing. In a **min-heap**, the smallest element has highest priority at the root. In a **max-heap**, the largest element is at the root.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

```
pq.offer(40);
pq.offer(10);
pq.offer(30);
```

```
System.out.println(pq.peek()); // 10
System.out.println(pq.poll()); // 10
```

- Insert into a binary heap:  $O(\log n)$ .

- Remove root:  $O(\log n)$ .
- Peek root:  $O(1)$ .
- Build heap from  $n$  elements can be done in  $O(n)$  with bottom-up heap construction.
- Java's PriorityQueue is a min-priority queue by default.

```
PriorityQueue<Integer> maxHeap =  
    new PriorityQueue<>(Collections.reverseOrder());
```

## 6. Hashing in Java

Hashing maps a key to a location using a hash function. Java provides HashMap, HashSet, and related collection classes.

### HashMap

```
HashMap<String, Integer> marks = new HashMap<>();

marks.put("Java", 90);
marks.put("Python", 85);

System.out.println(marks.get("Java"));
System.out.println(marks.containsKey("Python"));

for (Map.Entry<String, Integer> entry : marks.entrySet()) {
    System.out.println(entry.getKey() + " = " + entry.getValue());
}
```

- Stores key-value pairs.
- Keys are unique; putting the same key again replaces its associated value.
- Average-case lookup, insertion, and deletion are commonly treated as  $O(1)$ , subject to hashing and implementation details.
- HashMap does not guarantee insertion order.

### HashSet

```
HashSet<Integer> set = new HashSet<>();

set.add(10);
set.add(20);
set.add(10);

System.out.println(set);
```

HashSet stores unique elements and is useful for duplicate detection and membership testing.

## 7. Graphs

A graph consists of vertices (nodes) and edges (connections). Graphs can be directed or undirected, weighted or unweighted.

### Adjacency List in Java

```
ArrayList<ArrayList<Integer>> graph =
    new ArrayList<>();

int n = 5;

for (int i = 0; i < n; i++) {
    graph.add(new ArrayList<>());
}

graph.get(0).add(1);
graph.get(0).add(2);
graph.get(1).add(3);
```

### BFS – Breadth First Search

```
void bfs(ArrayList<ArrayList<Integer>> graph, int start) {
    boolean[] visited = new boolean[graph.size()];
    Queue<Integer> q = new LinkedList<>();

    q.offer(start);
    visited[start] = true;

    while (!q.isEmpty()) {
```

```

        int u = q.poll();
        System.out.print(u + " ");

        for (int v : graph.get(u)) {
            if (!visited[v]) {
                visited[v] = true;
                q.offer(v);
            }
        }
    }
}

```

## DFS – Depth First Search

```

void dfs(int u,
        ArrayList<ArrayList<Integer>> graph,
        boolean[] visited) {

    visited[u] = true;
    System.out.print(u + " ");

    for (int v : graph.get(u)) {
        if (!visited[v]) {
            dfs(v, graph, visited);
        }
    }
}

```

- BFS uses a queue and explores level by level.
- DFS uses recursion or an explicit stack and explores deeply before backtracking.
- For adjacency-list representation, BFS and DFS are  $O(V + E)$ .



## 8. Searching

### Linear Search

```
int linearSearch(int[] a, int target) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == target)
            return i;
    }
    return -1;
}
```

Time complexity:  $O(n)$ . It works on both sorted and unsorted arrays.

### Binary Search

Binary search requires the data to be sorted. It repeatedly halves the search interval.

```
int binarySearch(int[] a, int target) {
    int low = 0, high = a.length - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (a[mid] == target)
            return mid;
        else if (a[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}
```

Time complexity:  $O(\log n)$ .

## 9. Sorting

### Bubble Sort

```
void bubbleSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        for (int j = 0; j < a.length - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}
```

Average/worst-case:  $O(n^2)$ .

### Insertion Sort

Builds the sorted portion one element at a time. Average/worst-case:  $O(n^2)$ ; best case:  $O(n)$  for already sorted data.

### Merge Sort

Divide-and-conquer sorting algorithm with  $O(n \log n)$  time and additional  $O(n)$  auxiliary space for the usual array implementation.

### Java Built-in Sorting

```
import java.util.Arrays;

int[] a = {5, 2, 8, 1, 3};
```

```
Arrays.sort(a);  
System.out.println(Arrays.toString(a));
```

## 10. Java Collections Framework

Java Collections Framework provides reusable interfaces and classes for common data structures.

| Interface / Class | Purpose                                | Typical Choice          |
|-------------------|----------------------------------------|-------------------------|
| List              | Ordered collection, duplicates allowed | ArrayList / LinkedList  |
| Set               | Unique elements                        | HashSet / TreeSet       |
| Map               | Key-value pairs                        | HashMap / TreeMap       |
| Queue             | FIFO / processing order                | ArrayDeque / LinkedList |
| Deque             | Insert/remove from both ends           | ArrayDeque              |
| PriorityQueue     | Priority-based removal                 | PriorityQueue           |

### ArrayList vs LinkedList

- ArrayList is generally preferred for frequent indexed access.
- LinkedList can be useful when operations at its ends are central, but it is usually not the first choice for random access.
- Choose based on the access and update pattern rather than assuming one is always faster.

### ArrayDeque

```
Deque<Integer> deque = new ArrayDeque<>();
```

```
deque.addFirst(10);  
deque.addLast(20);
```

```
System.out.println(deque.removeFirst());  
System.out.println(deque.removeLast());
```

## 11. Quick Complexity Revision

| Data Structure / Algorithm | Access | Search   | Insert   | Delete   |
|----------------------------|--------|----------|----------|----------|
| Array                      | O(1)   | O(n)     | O(n)     | O(n)     |
| Singly Linked List         | O(n)   | O(n)     | O(1)*    | O(1)*    |
| Stack                      | O(n)** | O(n)**   | O(1)     | O(1)     |
| Queue                      | O(n)** | O(n)**   | O(1)     | O(1)     |
| HashMap (average)          | —      | O(1)     | O(1)     | O(1)     |
| BST (average)              | —      | O(log n) | O(log n) | O(log n) |
| Heap                       | —      | O(n)     | O(log n) | O(log n) |

\* O(1) when inserting/deleting at a known position/node reference; finding that position may take O(n). \*\* Stack/queue do not provide efficient random access as their primary operation.

### Exam-Focused Points

- Remember: Stack = LIFO, Queue = FIFO.
- BST inorder traversal gives sorted order.
- Binary search needs sorted data.

- HashMap stores key-value pairs; HashSet stores unique values.
- PriorityQueue gives priority-based access; Java's default is a min-heap behavior.
- BFS uses Queue; DFS uses recursion/Stack.
- Know the difference between array and linked list.
- Practice writing insert, delete, search, traversal, and sorting programs without looking at notes.

## 12. Practice Programs

- Write a Java program to insert and delete elements in an array.
- Implement a singly linked list with insertion at beginning and end.
- Implement stack using an array.
- Implement queue using an array.
- Implement a BST and print inorder, preorder, and postorder traversals.
- Find the minimum and maximum element in a BST.
- Implement linear search and binary search.
- Implement bubble sort and insertion sort.
- Count frequency of elements using HashMap.
- Remove duplicates from an integer array using HashSet.
- Implement BFS and DFS using an adjacency list.
- Use PriorityQueue to process elements in increasing order.

**Study order:** Arrays → Linked List → Stack → Queue → Trees/BST → Heap → Hashing → Graph → Searching → Sorting → Collections.

This order builds the concepts step by step and also prepares you for Java coding interviews and data-structures practice.